

AI Library Platform

NHN 아카데미 KDT 3기



목차

01	프로젝트 소개	3p
02	검색	6p
03	캐싱	9p
04	리뷰	11p
05	텔레그램 & MCP	13p
06	코드설명	17p
07	자체 평가 의견	26p

프로젝트 개요

Tech Stack

Backend	Java 21, Spring Boot 3.5
Data Access	Spring Data JPA, QueryDSL
Database	PostgreSQL with pgvector, Redis
AI/LLM	Spring AI, Google Gemini, Ollama, BGE-M3
Batch	Spring Scheduler
Frontend	Thymeleaf

주요 기능

- 키워드 검색
- 벡터 기반 의미 검색
- 하이브리드 검색 (RRF)
- RAG 기반 AI 답변 생성
- 캐싱
- 리뷰 시스템
- Telegram AI Bot & MCP Tool



팀 구성 및 역할

고나영



MCP, Telegram

Telegram Bot
& MCP Tool

김태희



Review

도서 리뷰 관리 서비스
MCP Tool

윤채영



Search

벡터 및 하이브리드 검색
시멘틱 캐싱
MCP Tool

정은선

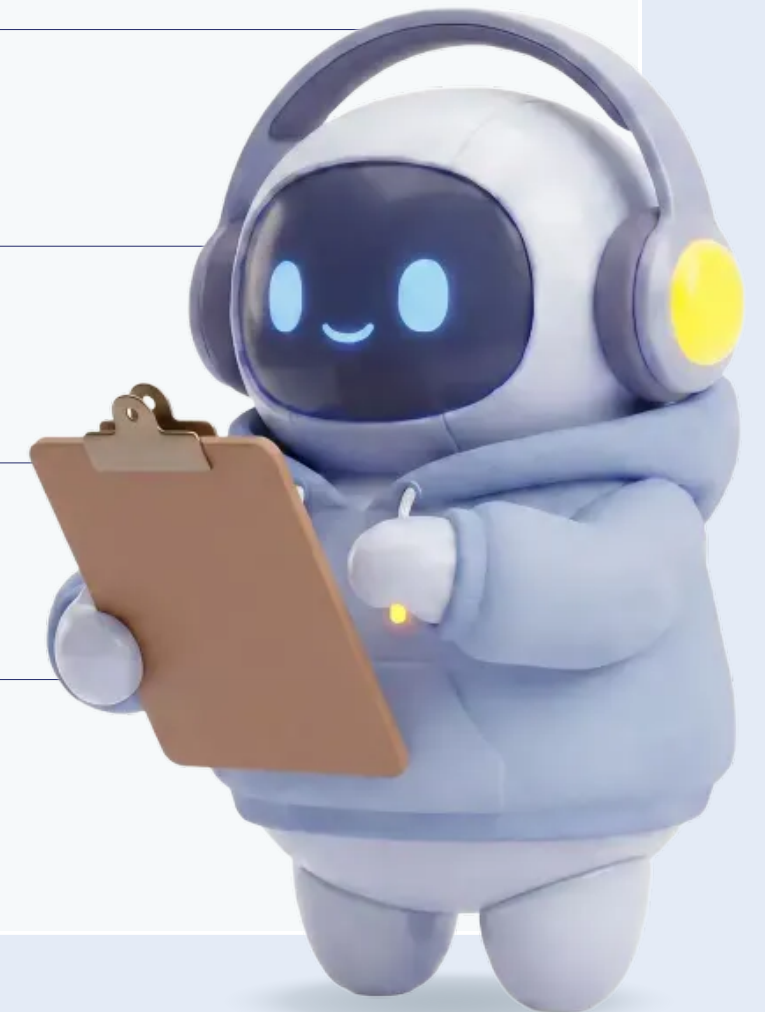


Search

RAG 검색
Redis 캐싱

프로젝트 수행 절차

- ✓ 06.22. → CSV 배치 적재
- ✓ 06.23. ~ 06.24. → 배치 적재 코드 리뷰 및 LIKE/인덱스 기반 검색 구현
- ✓ 06.25. ~ 06.28. → 벡터 검색, RAG 검색, 리뷰, Telegram 1차 구현
- ✓ 06.29. ~ 07.02. → 1차 구현 코드 리뷰 및 2차 구현 (코드 개선 / 성능 향상 / 남은 기능 구현)
- ✓ 07.03. → 발표, 보고서 작성





검색



Keyword

일반적인 키워드 검색과 전문 검색을
결합한 검색 기능



Vector

임베딩한 벡터값을 기반으로
코사인 유사도를 계산해 검색



Hybrid

키워드 검색과 벡터 검색 결과를 조합해
순위를 매기는 하이브리드 검색



RAG

하이브리드 검색의 상위 결과를 기반으로
의미 기반 재정렬 및 답변을 생성하는 검색 증강

검색 유형

KeyWord 검색 및 Vector 검색

KeyWord

키워드 검색 일반적인 키워드 검색과 전문 검색

- DB쿼리의 LIKE 문법을 사용한 일반적인 키워드 검색
- PostgreSQL의 전문검색 함수 (@@)를 이용해 전문 검색기능 추가

Vector

벡터 검색 키워드(검색어)의 벡터 및 책 정보 벡터의 코사인 계산을 통한 벡터 검색

- 스케줄러를 통해 5초마다 Book테이블에서 embedding이 NULL인 책을 여러 권 조회 후 일괄 임베딩 후 저장 (배치)
- 저장한 임베딩 벡터값과 키워드의 벡터값을 코사인 유사도 계산을 통해 임계값 필터로 검색

검색 유형

Hybrid, RAG

Hybrid

하이브리드 검색

키워드 검색과 벡터 검색을 조합한 검색

- 키워드 검색과 벡터 검색을 스레드 풀을 이용해 병렬로 실행 (Java CompletableFuture 컨테이너)
- 두 검색이 완료된 후 반환된 결과를 통해 순위를 기준으로 RRF 점수 계산
- RRF 점수 계산
 - $1 / (k + \text{순위} + 1)$ (k는 상수, $k = 60$) => 순위로 인한 값 변동 범위를 줄이기 위해 적당한 크기의 상수 사용

RAG

RAG 검색

의미 기반 재정렬 및 답변을 생성하는 검색 증강

- 하이브리드 검색을 통해 RAG용 후보 데이터를 추출
- 선택된 데이터와 리뷰 요약, 평점 정보를 기반으로 컨텍스트 구성
- 구성된 컨텍스트를 활용해 LLM 기반 질의 응답 생성 (검색 증강)



캐싱



키 매칭기반 조회

일반적인 키워드 검색과 전문 검색을
결합한 검색 기능



유사도 기반 조회

임베딩한 벡터값을 기반으로
코사인 유사도를 계산해 검색

캐싱 전략

키 매칭기반 조회

동일 Key 요청 시 Redis 캐시 데이터를 반환하여
조회 성능 향상
@Cacheable을 적용하여 캐시 저장 및 조회를
선언적으로 처리
검색 유형별 Prefix 분리(Keyword, ISBN)로 캐시
Key 충돌 방지 및 관리성 향상

유사도 기반 조회

검색어(KeyWord)를 벡터화하여 검색 결과와 같이 캐
시 저장
검색 시 검색어를 벡터화하여 캐싱되어있는 벡터화 검
색어와 코사인 유사도를 계산해 유사여부 판단
유사시 캐시 데이터 반환, 유사하지 않을 시 새로운 캐
시 저장



리뷰



비동기 메시지 큐

리뷰 등록 시 서버 부하를 방지하기 위해
RabbitMQ 기반의 비동기 파이프라인 구축



비용 최적화

새 리뷰 추가 시 누적 요약하는 방식으로
LLM API 토큰 비용을 절감



동시성 제어

동일 도서에 대한 중복 접근을 막기 위해
Redis 분산 락(Redisson)을 적용하여 정합성을 보장



영속성 확보

PostgreSQL에 요약본을 영구 저장해
데이터 영속성을 확보

도서 리뷰 등록 흐름

Step 01

Step 02

Step 03

Step 04

Step 05

리뷰 등록

사용자가 리뷰를 작성하면
book_review 테이블에
데이터 저장

비동기 작업 큐 적재

내부 이벤트 발행 후
RabbitMQ 큐에
'리뷰 요약 및 통계' 작업
비동기 메시지로 넣음

백그라운드 작업 소비

대기중인 백그라운드
워커 스레드들이
큐에서 작업을 순차적으로
꺼내 처리 시작

동시성 제어

동시에 같은 책의
요약 작업이 실행되는 것을
막기 위해 Redis 분산 락을
시도하여 락을 획득한 스레
드만 작업을 수행

AI 요약

LLM을 호출해 요약을 생성,
최종 결과를
book_review_summary
테이블에 저장 후 락 해제



리뷰 화면

리뷰 통계



AI 독자 평판 요약

장점:

- 제주 4·3 사건이라는 비극적 역사를 몽환적이고 시적인 문체로 형상화하여, 독자가 타인의 고통을 깊이 공감하게 만드는 강력한 서사적 힘을 지녔다.
- 새, 눈, 통나무 등 상징적인 소재를 활용해 인물들의 내면과 역사의 아픔을 입체적으로 표현했으며, 고통을 마주하고 치유해 나가는 과정을 우아하고 깊이 있게 그려냈다.
- 단순히 역사적 사실을 나열하는 데 그치지 않고, 인물들의 지극한 사랑과 그리움을 통해 기억의 영원성을 감동적으로 담아냈다.

단점:

- 현실과 꿈의 경계를 오가는 복잡한 서사 구조로 인해 초반부에는 이야기 전개가 느리게 느껴지거나 몰입하기 어려울 수 있다.
- 역사적 트라우마를 적나라하고 사실적으로 묘사하여 읽는 과정에서 심리적 피로감이나 무력감을 느낄 수 있다.
- 사건의 명확한 해결보다는 여운을 남기는 결말 방식으로 인해 독자에 따라 호불호가 갈릴 수 있다.

총평:

- 잊혀진 역사의 상흔을 기억하고 그 고통과 끝내 작별하지 않겠다는 작가의 결연한 의지가 담긴 작품으로, 한강 작가 특유의 깊은 통찰과 문장력이 돋보이는 수작이다.

추천 대상:

- 한국 현대사의 아픈 기억을 문학적으로 깊이 있게 성찰하고 싶은 독자나, 한강 작가 특유의 서정적이면서도 강렬한 문체를 통해 기억의 연대를 고민하는 이들에게 추천한다.

이 요약은 실제 독자들의 리뷰를 바탕으로 AI가 생성한 정보입니다.

리뷰 통계

사용자가 입력한 별점 데이터를 실시간으로 수집하여
평균 별점과 분포도를 보여줌

AI 요약

사용자가 입력한 리뷰를 LLM으로 압축하여 장점,
단점, 총평, 추천 대상을 보여줌



텔레그램 & MCP



Telegram Bot

사용자와 상호작용하는 채팅 인터페이스
Agent로부터 받은 응답 값 format 처리



Agent

자연어를 분석하고 적절한 Tool을 선택
만들어진 응답 값 Telegram에 전달



MCP

외부 기능을 표준 방식으로 연결 및 실행



Open API

공공 도서·도서관 데이터 조회

텔레그램 메시지 흐름

Step 01

Step 02

Step 03

Step 04

Step 05



Telegram

메시지 수신
/agent/chat에 전달

Agent

파라미터 추출
LLM에게 전달
MCP Tool 호출 명령



MCP Server

정보나루 API와 연동
응답 객체 생성

Agent

Tool 호출 결과를
Telegram에 전달



Telegram

Formatter 처리를 통해
요청 유형 별 일관된
응답 메시지 전송

채팅 결과

텔레그램 & MCP 서비스에서 지원하는 기능 및 결과입니다.

Question

- ✓ 대출 가능 도서관 정보 조회 (param : 지역, 세부지역, 도서명)
- ✓ 도서 정보 조회 (param : 도서명, 저자명)
- ✓ 인기 대출 도서 정보 조회 (param : 성별, 연령, 도서명, 지역 등)
- ✓ 이달의 키워드 조회 (param : 날짜)



도서 대출 가능 도서관 조회

광주 남구에서 소년이 온다 대출 가능한 도서관 오전 8:48 ✓

검색 중입니다. 잠시만 기다려주세요. 오전 8:48

대출 가능 도서관 검색 결과

[광주광역시립사직도서관]
주소 : 광주광역시 남구 제중로 56
전화번호 : 062-613-7783
홈페이지 : <https://citylib.gwangju.go.kr/main>
휴관일 : 매월 첫째주, 셋째주 월요일 / 관공서 공휴일, 도서관 사정으로 관장이 지정한 날
운영시간 : 종합자료실 09:00~22:00(주말 17:00), 어린이실 09:00~18:00(주말 17:00), 열람실 07:00~22:00
소장 여부 : 소장 중
대출가능 여부 : 대출 가능

[광주남구청소년도서관]
주소 : 광주광역시 남구 제석로 100
전화번호 : 062-607-2546
홈페이지 : <https://lib.namgu.gwangju.kr/>
휴관일 : 매주 첫째주, 셋째주 월요일 / 법정공휴일
운영시간 : 청소년자료실: 평일 09:00~22:00, 주말 09:00~17:00 / 어린이자료실: 평일 09:00~18:00, 주말 09:00~17:00
소장 여부 : 소장 중
대출가능 여부 : 대출 가능

Troubleshooting - RAG

문제

- 검색 결과 전체 Context 전달
- 관련성이 낮은 도서 포함
답변 품질 저하 및 토큰 사용량 증가

해결방법

- Hybrid Search Score 기준 정렬
- 상위 Top-10 Context 구성
- 리뷰 요약·평균 별점 추가
- LLM Context 전달

결과

- Context 노이즈 감소
- 답변 관련성 및 품질 향상
- 토큰 사용량 절감

Troubleshooting - Redis 캐시 최적화

문제

- 동일한 검색 요청에도 매번 DB 조회 수행
- Redis API를 직접 호출하여 캐시 로직이 서비스 코드에 분산
- 검색 유형별 Cache Key 충돌 가능

해결방법

- @Cacheable을 적용하여 캐시 저장/조회 자동화
- Keyword, ISBN별 Prefix로 Cache Key 분리
- Redis를 활용한 검색 결과 캐싱

결과

- 반복 조회 응답 속도 향상
- Cache Key 충돌 방지
- 캐시 관리 코드 간소화

Troubleshooting - Semantic Cache

문제

- 동일한 의미의 질문도 Cache Miss 발생
- 매 요청마다 DB 조회 및 LLM 호출 수행
- 응답 시간이 증가

해결방법

- 질문을 임베딩(Vector)으로 변환
- Redis Vector Search(KNN) 수행
- 유사도 기준 이상이면 Cache Hit
- Cache Miss 시 신규 Cache 저장

결과

- 의미가 같은 질문도 Cache Hit
- 반복적인 DB 호출 감소
- 응답 속도 향상 및 비용 절감

RAG 검색 흐름



RAG Context 구성 코드

top-10 추출

- Score 기준 상위 10건 선택
- Context 노이즈 감소

Context 정보 보강

- 리뷰 요약 추가
- 평균 평점 추가
- LLM 이해도 향상

```
List<BookSearchResponse> books = hybridResult.stream()
    .limit(maxSize: 10)
    .toList();
log.info("검색된 도서 수: {}", books.size());
```

```
for (BookSearchResponse book : books) {
    BookReviewSummary summary = summaryMap.get(book.getId());
    //리뷰 요약
    String reviewSummary = summary != null
        ? summary.getReviewSummary()
        : "리뷰 없음";

    //별점 평균
    BigDecimal averageRating = summary != null
        ? summary.getAverageRating() : BigDecimal.ZERO;

    sb.append("제목: ").append(book.getTitle()).append("\n")
        .append("내용: ").append(book.getContent()).append("\n")
        .append("유사도: ").append(book.getSimilarity()).append("\n")
        .append("리뷰요약: ").append(reviewSummary).append("\n")
        .append("별점평균: ").append(averageRating).append("\n")
        .append("isbn: ").append(book.getIsbn()).append("\n")
        .append("\n");
}
```

Redis를 활용한 조회 성능 최적화

- @Cacheable 적용
- Keyword/ISBN Prefix 분리
- Cache Key 충돌 방지

```
@Override @Jungeunsun565
@Cacheable(
    value = "team3:keyword",
    key = "#request.searchType() + ':' + #request.keyword().trim().toLowerCase()"
)
public List<BookSearchResponse> search(BookSearchRequest request) {
    return bookRepository.searchByKeyword(request.keyword());
}
```

```
@Override @Jungeunsun565
@Cacheable(
    value = "team3:isbn",
    key = "#request.searchType() + ':' + #request.isbn()"
)
public List<BookSearchResponse> search(BookSearchRequest request) {
    return bookRepository.searchByIsbn(request.isbn());
}
```

Redis Semantic Cache 구현

Redis Vector Index 생성

- JSON 기반 Vector Index 생성
- 임베딩 벡터 검색 환경 구성

```
FT.SEARCH team3_semantic_cache_index
"*⇒[KNN 10 @embedding $vec AS score]"
PARAMS 2 vec <binary_vector>
SORTBY score
RETURN 3 $.question $.embedding score
DIALECT 2
```

Redis Semantic Cache 구현

Redis 캐싱 저장

- 검색어 Key 정규화
- 질문 임베딩(Vector) 생성
- SemanticCache DTO 구성
- JSON 직렬화
- Jedis를 이용한 Redis JSON 저장

```
@Override 1개 사용 위치  Ⓜ Jungeunsun565
public void put(SearchType type, String keyword, String context, List<BookSearchResponse> fullResult) throws JsonProcessing
    String normalized = keyword.trim().toLowerCase();
    String key = String.format("team3:semantic:%s:%s", type, normalized);

    float[] embeddedContext = embeddingService.getEmbedding(text: context + keyword);

    SemanticCache semanticCache = new SemanticCache(keyword, type, embeddedContext, fullResult, (long) fullResult.size());

    String json = objectMapper.writeValueAsString(semanticCache);

    jedis.jsonSet(key, json);
    jedis.expire(key, seconds: 3600);
}
```

Redis Semantic Cache 구현

Redis 캐싱 조회

- 질문 임베딩 생성
- FT.SEARCH KNN으로 최근접 이웃 검색
- 거리(Score) 기준 Top-10 조회
- Threshold 이내의 결과만 Cache Hit 처리

```
try (Socket socket = new Socket(host: "220.67.216.14", port: 6379)) {  
  
    socket.setSoTimeout(5000);  
  
    OutputStream out = socket.getOutputStream();  
    InputStream in = socket.getInputStream();  
  
    byte[] vec = floatArrayToBytes(embeddedContext);  
  
    //쿼리 바이트로 생성  
    List<byte[]> args = List.of(  
        "FT.SEARCH".getBytes(),  
        "team3_semantic_cache_index".getBytes(),  
        "*=>[KNN 10 @embedding $vec AS score]".getBytes(),  
    );  
}
```

```
if (score > THRESHOLD) {  
    return List.of();  
}
```

자체 평가 의견

01

잘한 점

- Hybrid Search 기반 RAG 구현
- Redis Semantic Cache 적용
- Socket 통신 기반 Redis FT.SEARCH 호출 구현

02

아쉬운 점

- Threshold와 Top-K 값이 경험적으로 결정되어 정량적인 성능 평가가 부족
- Redis Semantic Cache를 API를 활용하지 못함

03

배운 점 및 개선 사항

- Redis Cache와 Semantic Cache를 적용하며 캐시 전략의 중요성을 학습
- 사용자에게 더욱 일관되고 안정적인 검색 결과를 제공하도록 검색 성능 고도화



May 25, 2099

AI Literacy

Smart Tools, Smart Users

